



A TRIDENT FORGE FIELD GUIDE

THE PERSISTENT MEMORY BUNDLE

*The file-based memory discipline I run under Claude
Code, Cowork, and the Agent SDK — packaged so you
can drop it in today.*

THOMAS ADAIR

If you read the essay this ships with, you already have the argument: memory does not belong inside the model. It belongs in files you own, structured on purpose, updated on a discipline, and read by every session that opens. This guide is the working version of that idea. Not the philosophy — the moves. Where the essay says *write things down at the moment they happen*, this tells you what the file looks like, what goes in the front-matter, how the index stays readable, and where the thing rots if you stop tending it.

I built this for myself first. It has carried a trading system, an audio-plugin line, an ebook project, and a pile of research across model rotations and long gaps. What you are dropping in is the same skeleton I run, stripped of my data and rebuilt as something a stranger can use. Everything here is a pattern with an example beside it. Take the pattern; throw the example away the moment your own content replaces it.

01 · THE PROBLEM

WHAT THE BUNDLE IS, AND WHY FILES WIN

WHAT A MEMORY BUNDLE IS

What a memory bundle is — and what it is not

A memory bundle is a directory of small Markdown files, each holding one fact, plus a single index file that maps them. That is the whole shape. A file per fact, a line per file in the index, and a discipline that keeps both current.

It is *not* a database. It is not a vector store, not an embeddings pipeline, not a graph engine with a query planner. It is not a chat history you scroll back through, and it is not the model's context window dressed up with a longer token limit. Those things solve retrieval-at-scale problems you do not have yet, and they solve them by making the memory illegible to you — you cannot open an embedding and read it, and you cannot hand-edit a similarity score.

The bundle solves a different problem: keeping a small, high-value set of durable facts *readable by the next session and by you*. Who you are. How you want the model to work. What each live project's current truth is. Which papers and repos you want kept close, and why. That set is small — it fits in a couple hundred files, most of them under a page — and the whole point is that it survives the death of the session that wrote it.

One more thing it is not: a log. A log grows forever and nobody reads the middle. A memory bundle is pruned. A memory that went stale gets corrected or retired, not archived. If you find yourself never deleting anything, you are keeping a log, and the index will drown.

WHY FILE-BASED BEATS A VECTOR STORE FOR THIS

Why file-based beats a vector store for this

This is the part people push back on, so let me tag my claims so you can weigh them yourself. I use three labels through this guide: **VERIFIED** (I have watched it hold in practice), **INFERRED** (it follows from how these systems work, and matches what I have seen), **GUESS** (I believe it but have not proven it).

The read is free and the read is legible. [VERIFIED] A Markdown file opens in the session's first request with no infrastructure — no embedding call, no vector index to warm, no service to be up. The model reads the actual words you wrote, not the three nearest neighbors to a query vector. When the memory is wrong, you open the file and fix the sentence. You cannot do that to a vector store without re-embedding, and you usually cannot even *see* what it decided to return.

Precision beats recall at this scale. [INFERRED] Vector search earns its keep when you have thousands of documents and no idea which three matter. At a couple hundred deliberately-authored files, you already know which one matters — it is named

`project_<the-thing-you-are-working-on>.md`, and the index points you straight at it.

Semantic retrieval over a small curated set mostly adds a failure mode: it surfaces the plausibly-related instead of the actually-relevant, and it does it silently. Grep does not do plausible. Grep finds the string or it does not.

Ownership and portability. [VERIFIED] The bundle is plain text on a filesystem. It moves between machines with `rsync`, lives in a git repo if you want history, and is readable in ten years by anything that reads text. A vector store is a running service with a schema and a version, and when that service changes its embedding model your old vectors are subtly incomparable to your new ones. [INFERRED] Text has no such drift.

Where the vector store wins. [VERIFIED] If your memory is truly large — a research corpus of thousands of documents you cannot name in advance, transcripts you want to search by meaning rather than keyword — then yes, you want embeddings, and the two are not enemies. The move there is to keep the *curated* bundle as your source of truth and point a vector index at the *bulk* corpus as a search surface over it. The bundle stays small and legible; the index handles scale. This guide is about the bundle. If you outgrow it, you will know, because grep will start missing things you know are in there.

The honest summary: file-based memory is the right default because the read is free, the content is legible, the ownership is total, and at this scale precision matters more than recall. Reach for a vector store when the corpus outgrows what you can name — not before.

02 · THE SHAPE

TYPES, NAMING, THE INDEX, AND THE GRAPH

THE FOUR MEMORY TYPES

The four memory types

Every file in the bundle is one of four types, declared in its front-matter. The type is not decoration — it tells the next session how to weigh the file.

User memories hold facts about who you are. Your role, your domain, your standing preferences for how you want to be addressed and how output should be shaped. These change rarely and they anchor everything else. A session that reads a user memory knows the register to reach for before it writes a word.

Feedback memories hold behavioral rules — corrections you have given the model that you do not want to give again. This is the type that compounds hardest. Every time you catch yourself explaining the same preference twice, the second time is a bug in the bundle, and the fix is a feedback file. Over months these accumulate into a working style the model can load instead of rediscover.

Project memories hold the current state of a build in flight. One file per project. Not the history of the project — the *current truth* of it: what it is, where it stands, what is decided, what is open. When a project decision locks, the project file changes the same day, or it starts to lie.

Reference memories hold pointers to things outside the bundle — a paper, a repo, a thread — with your note about *why* it matters. The why is the load-bearing part. A bare bookmark is worthless in six weeks; you will not remember why you saved it. A reference memory that says “this paper’s section 4 is the argument I want for the retry-logic decision” is a tool you can use.

Each type has a canonical shape. Front-matter carries a `name` (a short kebab-case slug), a one-line `description` that is rich enough to decide relevance from the index alone, and a `metadata` block declaring the `type`. The body reads like a note from you to a future you — because that is exactly what it is. The four `templates/example_*.md` files ship one worked example of each, built on invented personas. Read those next to this section; they are the pattern in the concrete.

Naming conventions and slug hygiene

The filename is the first thing a future session sees, so it does work. The convention is

`<type>_<slug>.md` :

- `user_dana_backend_engineer.md`
- `feedback_run_tests_before_done.md`
- `project_invoice_sync_service.md`
- `reference_idempotency_keys_stripe_docs.md`

The `type_` prefix means a directory listing sorts into its four families and a `ls feedback_*` shows you every rule at once. The slug is lowercase, underscore-separated, and specific enough to be unique — `project_invoice_sync_service`, not `project_service`. If you find yourself wanting to name two files nearly the same thing, that is a signal the two facts want to be one file, or the slugs need a distinguishing word.

Inside the front-matter, the `name` field mirrors the slug in kebab-case (`invoice-sync-service`), because that is the form the cross-link convention uses. Keep the filename slug and the `name` field in lockstep — same words, one with underscores for the filesystem, one with hyphens for the links. When they drift, links break quietly.

Dates in filenames only when the file is inherently a point-in-time artifact — an audit, a session handoff, a dated decision you will want to find by date. A `project_` or `feedback_` file is a *living* file; it gets updated in place and does not carry a date in its name, or you will end up with five dated copies and no source of truth.

MEMORY.MD AS A SLIM INDEX

MEMORY.md as a slim index — the cap discipline

The index is the concierge. It is the one file every session reads first, and its entire job is to be small enough to read in one shot and point at everything else.

The format is one line per entry: `[Title](file.md) – one-sentence hook`. Grouped under headers by type and by project area. Nothing more. The index does not try to *hold* the memories — it holds the *map*. The moment it starts carrying content, it stops being readable, and an unreadable index is worse than no index, because the tail entries silently fall out of the model's read window and the session proceeds as if they do not exist.

This is the failure mode to watch: **the index grows past the point where it reads in a single request, the last entries drop off the bottom, and you never notice** — because nothing errors. The session just quietly stops seeing your newest memories. I cap the index around two hundred lines and compact when it crosses. Compacting is mechanical: collapse related entries onto shared lines, retire dead ones, push detail down into the files where it belongs. It is taking out the trash. It is not interesting and it has to be done, and the day you skip it is the day a session forgets something you told it last week.

A slim index is also how the read stays cheap. A light, fast model can open a two-hundred-line index, find the one relevant line, and open the one relevant file — three cheap reads instead of dragging the whole bundle into context. The cap is not tidiness for its own sake. It is what keeps the whole system loadable.

CROSS-LINKING AS A LIGHT KNOWLEDGE GRAPH

Cross-linking as a light knowledge graph

Files reference each other with a double-bracket slug: `[[feedback-run-tests-before-done]]`. It looks like a wiki link because it is one — the convention is borrowed from tools like Obsidian, and if you keep your bundle in one it renders as a real graph. But it earns its place even in plain text, because it turns a pile of files into a network with edges.

The edges are where the compounding shows up. A project memory links to the feedback rule that governs it. A feedback rule links to the reference paper that grounds it. A reference links to the sibling reference it sits beside in a debate. When a session opens one file and sees `[[...]]` slugs in the body, it has a map of what else is worth reading — not everything, just the neighbors. That is a knowledge graph you can hand-edit, that survives in plain text, and that never needs a query engine.

Two rules keep the links honest. Link liberally — a `[[slug]]` that points at a file you have not written yet is fine; it marks something worth writing, a gap in the graph, not an error. And keep the slug exactly matching the target's `name` field, because a link with a typo is a dead edge that looks live. When you rename a file, grep the bundle for the old slug and fix the edges. The graph is only as good as its links resolve.

03 · THE DISCIPLINE

WRITING, PRUNING, AND WHAT NEVER GOES IN

THE SAME-SESSION SAVE DISCIPLINE

The same-session save discipline

This is the rule that keeps the whole thing from lying, and it is the one I violated hardest before I made it non-negotiable.

OPERATOR NOTE

A memory gets written in the same session that produces the fact. Not later. Not “next time it comes up.” Not “I’ll grep the outputs folder and reconstruct it.” The moment a session produces something worth remembering — a locked decision, a publication-grade artifact, a correction, a reference worth a second look — the memory gets written before the session ends.

The failure that taught me this: a session produced a substantial piece of writing, dumped it to an outputs folder, and moved on without writing a project memory. Six days later I went looking for it, and memory had no idea it existed. The model offered me wrong candidates because the right one was invisible to it — present on disk, absent from memory. The artifact and its memory record are two surfaces of the same fact, and the drift between them was the bug.

The discipline generalizes past publication. Any of these, same session:

- A session corrects a class of behavior → a feedback file, before the session moves on.
- A project decision locks → the project file updates, same day.
- A paper or repo earns a second look → the reference gets saved now, because one you mean to save “next time” is one you have already lost.
- Every several substantive exchanges → checkpoint anything worth surviving, because sessions drop context without warning and the checkpoint is what catches it.

The reason it has to be *same session* is that the context which makes the memory worth writing is only present in the session that produced it. A day later you have the artifact but not the reasoning, the decision but not the alternatives you rejected. Write it while the context is live or write a worse version of it later.

WHEN TO UPDATE, WHEN TO WRITE NEW, WHEN TO P

When to update, when to write new, when to prune

Three operations, and picking the wrong one is how bundles rot.

Update in place when the *same fact* changed state. A project moved forward, a decision reversed, a rule got a new exception. The file already exists; open it, change the current-truth prose, and the history is gone on purpose — the file holds what is true now, not the diary of how it got there. A project memory is a photograph of the present, retaken, not a time-lapse.

Write new when it is a distinctly different fact. A new project, a new rule, a new reference. The test: would this share a filename with something that exists? If yes, it is an update. If no, it is a new file. Resist the urge to fold an unrelated fact into an existing file because it is “kind of related” — that is how files balloon into unreadable grab-bags. One file, one job.

Prune when a memory is wrong and will not come back. A rule that was correct in spring and wrong by summer does not get archived — it gets corrected if the corrected version is useful, or deleted if it is not. A dead memory is worse than a missing one, because a session will follow it dutifully and produce work that is confidently obsolete. The weekly pass is where pruning happens: read the bundle, find the memories that no longer match the world, and correct or kill them.

The balloon is the quiet failure here. A file that gets appended to every time it is “kind of relevant” grows until it no longer reads in one request, and then it has the same tail-drop problem as a bloated index — the bottom of the file stops being seen. Keep files single-purpose and small. If one is growing, that is usually two facts fighting to be separate files.

SENSITIVE-INFO EXCLUSIONS

Sensitive-info exclusions — concrete rules

The bundle is a file on your disk, but treat it as if it will be read by someone who is not you — because a session transcript can be shared, a screenshot can leak, and a bundle synced to a server has a wider blast radius than a local file. Some things never go in, regardless of instructional value.

Never write, in any memory:

- Passwords, API keys, tokens, or secrets of any kind. These live in environment variables and secret managers, never in a memory file. A memory can say “the API key is in the `PAYMENT_API_KEY` env var”; it never carries the value.
- Financial account numbers, credit details, or anything that identifies a specific account.
- Government IDs, home addresses, or other direct personal identifiers — yours or anyone else’s.
- Medical or health information about any named person.
- Named third parties in a way that exposes their private details. If a colleague’s working style matters to a rule, the rule can capture the *pattern* without the name.

The redaction rule when a fact is useful but carries something sensitive: **keep the method, drop the specifics**. A memory about a hard-won debugging lesson keeps the lesson and replaces the account, the customer, the internal system name with a generic placeholder — “the payment service,” “a downstream consumer,” “the reconciliation job.” The instructional value lives in the shape of the problem, not in whose data it was. When in doubt, write the placeholder. A slightly more abstract memory that is safe to leak beats a specific one that is not.

Run a periodic grep for the obvious tells — `key`, `token`, `password`, `secret`, an `@` for email addresses — as a cheap backstop. It will not catch everything, but it catches the careless paste.

04 · IN PRACTICE

WIRING IT UP, AND HOW IT ROTS

INTEGRATION

Integration — who loads MEMORY.md, and how

The bundle is portable across the tools that read a filesystem, but each one loads it a little differently. Here is the shape of it. [VERIFIED for the pattern; the exact wiring of any one tool changes, so confirm against current docs — INFERRED that the pattern holds across versions.]

Claude Code. The reliable path is an instruction file the tool reads at startup — a `CLAUDE.md` in the project or your home config — that tells the session to open `MEMORY.md` first and load relevant files from the bundle before doing anything else. You are not relying on the tool to magically find your memory; you are pointing it at the index with one instruction, and the index does the rest. Some setups add a session-start hook that reads the index automatically; the instruction-file path works everywhere.

Cowork / agent sessions. Same principle: a startup instruction that orients the session on the index. The bundle sits at a known path, the session opens the index, finds the relevant lines, and reads those files. When you run agents in parallel, they read the *same* bundle — which is a feature, because it is the shared source of truth that keeps parallel work coherent, and a caution, because two sessions writing the same file at once will clobber each other. Have parallel work write *different* files and reconcile at the index.

Claude Agent SDK. Here you wire it yourself, which means you control it completely. The pattern: on session construction, read the index file and inject it into the system prompt, then expose the bundle directory as a readable resource the agent can open files from. Because you own the loop, you can also enforce the discipline in code — a checkpoint step that writes a memory at the end of a run, a prune step on a schedule. The SDK is where the bundle stops being a habit and becomes machinery.

MCP and a server layer. If you want the bundle reachable from any session on any machine, put a small service in front of it that exposes read and query over an MCP interface. That is the “brain” layer from the essay — the local bundle stays fast and greppable, the server makes it portable, and if the server is down the session falls back to the local files. Build this only when you actually work across machines. On one laptop, the local bundle is the whole system.

The through-line: every integration is some version of *point the session at the index, let the index point at the files*. The tool changes; the move does not.

Common pitfalls

Over-indexing. Putting content in the index instead of pointers. The index balloons, the tail drops, and you have moved the rot from the files into the map. The index holds hooks, not facts.

Drift between memory and current state. The project moved and the project file did not. This is the most common rot and the most dangerous, because the file *looks* authoritative while being wrong, and a session trusts it. The same-session update rule is the fix; the weekly pass is the backstop.

Ballooning entries. A single file that grew a page too long because facts kept getting appended to it. The bottom stops being read. Split it, or prune it back to its one job.

Bookmark memories with no why. A reference that says what the thing is but not why you saved it. In six weeks it is noise. Every reference earns its place with a why.

Writing for now instead of for the stranger. A memory that only makes sense with the context you have this second. Read it back as if you have never seen it — if it does not stand on its own, it will not stand next month either.

Never pruning. Treating the bundle as append-only. It becomes a log, the signal drowns, and the discipline that was supposed to compound instead accumulates. Deleting is part of the practice, not a failure of it.

MEMORIES ARE POINT-IN-TIME

Memories are point-in-time — grep current state before you act

The last thing, and the one that matters most once the bundle is large: **a memory records what was true when it was written, not necessarily what is true now.**

Every file is a photograph with a timestamp. A project memory written three weeks ago describes the project three weeks ago. A feedback rule from spring may have a summer exception. A reference's "current version" is current as of the day you wrote it. The bundle is not a live view of the world — it is a stack of dated snapshots, and treating an old snapshot as live truth is how a session confidently acts on obsolete information.

So the discipline has a partner: **before you act on a memory, verify it against the current state of the actual thing.** If a memory names a file, confirm the file still exists before you recommend it. If it names a version, a path, a flag, a deadline — check the world, do not trust the snapshot. The memory tells you where to look and what mattered; the current state tells you what is true. When they disagree, the world wins, and the disagreement is itself a signal that the memory needs updating — same session.

This is why the whole system is built to be greppable and legible rather than clever and opaque. You can only verify a memory against reality if you can *read* the memory. A file you can open and check beats a vector you have to trust. The bundle does not remember for you. It remembers *with* you — and you are the one who checks its work.

That is the practice. A file per fact, a line per file, four types, a slim index, honest links, and a discipline that writes in the same session and prunes on a schedule. The templates beside this guide give you the skeleton; the examples show the shape. Drop them in, point your tool at the index, and write your first three memories today — a user file for who you are, a feedback file for the last thing you had to explain twice, and a project file for whatever you are building right now.

The rest compounds from there.

I did not invent this shape. Andrej Karpathy published a pattern for an LLM wiki — an index file, a set of small linked articles, wikilinks between them — and that gist is the direct ancestor of what this guide describes. Credit where credit is due.

Refunds and updates. Thirty days, no questions asked — email me and I refund it. Updates to this guide are free to buyers. The Cowork companion is a separate product.

— *Thomas Adair*